
NestFinder

Technical Documentation

A web platform helping Nigerian students find off-campus accommodation for their SIWES industrial-training placement.

Idris Bello

Final-year project · Abiola Ajimobi Technical University, Ibadan
July 2026

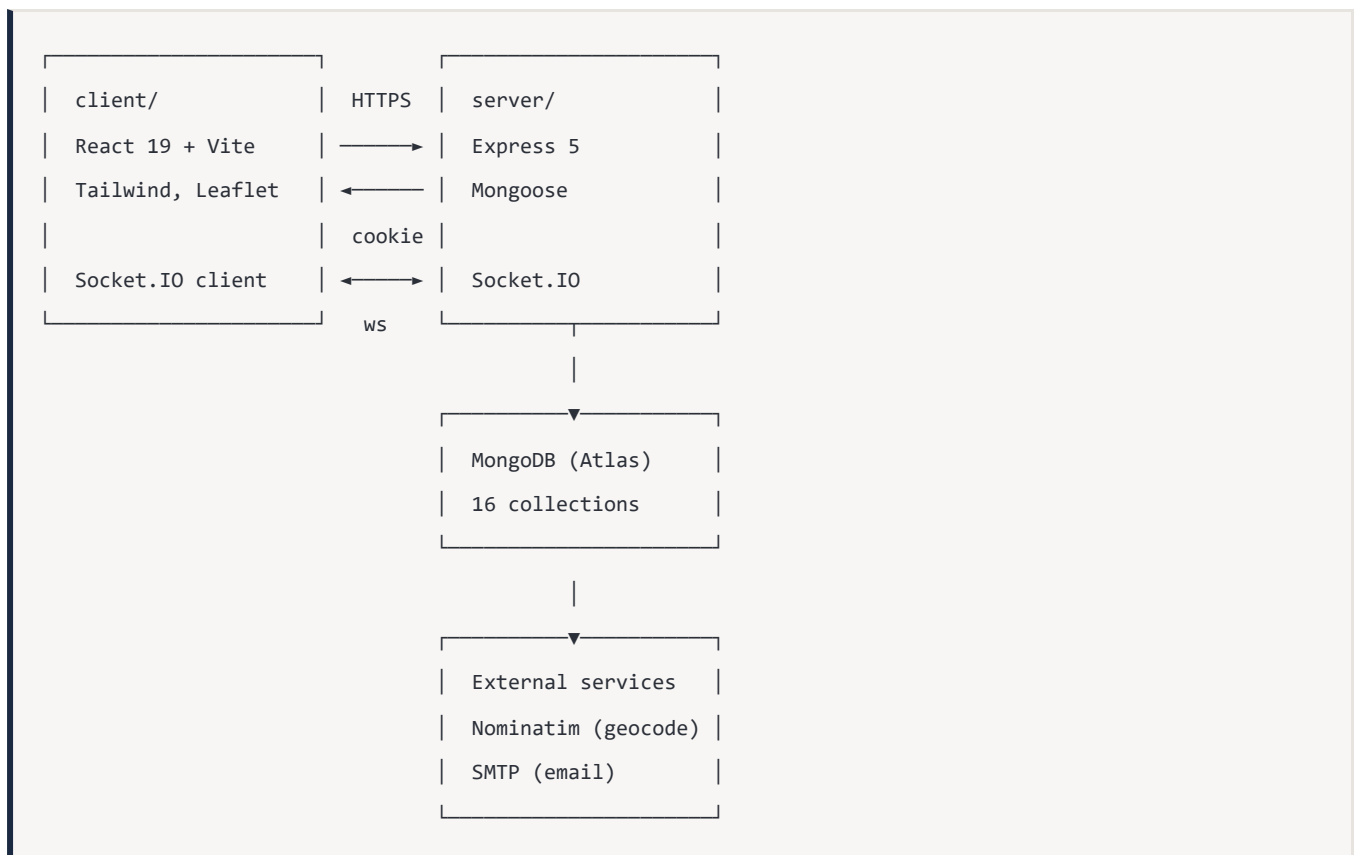
System design, data model, API reference and the reasoning behind the significant decisions.

Contents

- [Architecture](#)
 - [Authentication and sessions](#)
 - [Authorisation: the four gates](#)
 - [Data model](#)
 - [API reference](#)
 - [Location and geocoding](#)
 - [SIWES placement matching](#)
 - [Comparison scoring](#)
 - [Bookings and escrow](#)
 - [Fraud detection and trust](#)
 - [Security](#)
 - [Testing](#)
 - [Notable decisions](#)
-

Architecture

Two independently deployable applications sharing a MongoDB database.



Request flow: browser → Express middleware chain (security headers, CORS, rate limit, body parse, cookie parse) → route → guard middleware → controller → Mongoose model → MongoDB.

Server layering

LAYER	RESPONSIBILITY
<code>routes/</code>	URL shape and which guards apply
<code>middleware/</code>	Authentication, role checks, verification gates, uploads, validation
<code>controllers/</code>	Request handling, orchestration, responses
<code>services/</code>	Work that is not request-shaped: geocoding, email, image processing, fraud scanning, alerts
<code>models/</code>	Schemas, indexes, and model-level invariants
<code>utils/</code>	Pure functions: filter building, cost arithmetic, error reporting

Every external dependency sits behind a seam and degrades to a no-op when unconfigured. With no SMTP credentials the app prints emails to the console; with no CAPTCHA key the check passes through; with no error-reporting DSN, errors are still written locally. **The application always starts.** This matters for a project that must be demonstrable on a machine that has none of those keys.

Authentication and sessions

JWT in an httpOnly cookie. The token is signed with `JWT_SECRET` and set as `nf_token` (`middleware/auth.js`) with `httpOnly: true`, `secure` outside development, and `sameSite: 'none'` in production so the separately-hosted client can send it.

`httpOnly` means JavaScript cannot read it, so a cross-site scripting flaw cannot exfiltrate the session. The alternative — `localStorage` — is readable by any script on the page.

Login is two steps.

1. Email and password are checked. On success a six-digit one-time code is generated, hashed, stored with an expiry, and emailed.
2. The code is submitted; only then is a token issued.

A device that has recently completed step 2 can be remembered, which skips the code on that device only.

Email verification is a hard gate at login. An unverified account gets no token, no code, and no trusted-device shortcut — the check happens before any of that.

Session invalidation. `passwordChangedAt` is stored on the user. Any token issued before that timestamp is rejected, so changing a password logs out every other session immediately. Password reset tokens are stored hashed, single-use, and time-limited.

Brute-force protection is per email address, not per IP, so an attacker cannot dodge it by rotating IPs. Five failures inside ten minutes locks that address for thirty minutes. The lockout response is identical whether or not the password was correct, so it cannot be used to confirm a valid password.

Authorisation: the four gates

Guards compose left to right on a route. Understanding these four explains nearly every access rule in the system.

GUARD	QUESTION IT ANSWERS
<code>protect</code>	Is there a valid, non-expired token for a non-suspended account?
<code>studentOnly</code> / <code>adminOnly</code>	Is this the right kind of account?
<code>requireIdentityVerified</code>	Has an administrator approved this person's ID document?
<code>optionalAuth</code>	Attach the user if present, but do not require one

`optionalAuth` is what lets a listing page personalise for a signed-in student — showing distance to their placement — while still being fully public.

`requireIdentityVerified` **guards exactly four actions**, and the list is deliberately short:

- `POST /api/listings` — publishing a property

- `POST /api/listings/geocode-preview` — the address lookup used while drafting one
- `POST /api/messages/conversations` — starting a conversation
- `POST /api/bookings` — making a booking

The principle: **browsing is open, and the gate falls where a real person could be harmed**. Note that *starting* a conversation is gated but *replying* is not — a landlord must always be able to answer a student who has already reached out.

Data model

Sixteen collections. The core five:

Listing

The property. Alongside descriptive fields (`title` , `description` , `roomType` , `rooms` , `furnished`) it carries:

- **Pricing** — `price` with `priceUnit` (`year` or `month`), plus a derived `monthlyPrice` . Everything that compares or filters uses `monthlyPrice` , because a ₦50,000/month room and a ₦600,000/year room are the same price and must not sort differently.
- **Costs** — `cautionDeposit` , `agentFee` , `legalFee`
- **Location** — a GeoJSON `Point` with a sparse `2dsphere` index, plus `geocodePrecision` , `locationSource` and `locationConfirmedAt` recording *how* the coordinates were obtained and whether a human confirmed them
- **Living conditions** — nested `utilities` (electricity hours, water source, generator, internet, waste) and `environment` (noise, privacy, ventilation, lighting, surroundings)
- **Integrity** — `fraudScore` , `fraudLevel` , `flagged` , `reportCount`
- **Reputation** — `rating` , `totalReviews` , `views`

The location field defaults to `undefined` rather than being declared inline. A `2dsphere` index rejects a `Point` with no coordinates, so an inline default would make every listing without a location unsaveable.

Student

Identity and credentials, plus:

- `department` — determines which placement centres accept them
- `placement` — `{ company, role, startDate, endDate, status, confirmedAt }` . This subdocument is the anchor for all distance-based features.
- `placementNotice` — a submitted centre not yet in the directory
- `idDocument` — `{ documentType, frontImage, backImage, submittedAt, reviewedAt, status, rejectionReason }`

Implementation note. Mongoose materialises an empty `placement` subdocument, so `student.placement` is truthy for every student. Presence must be tested via `placement.company`, and confirmation via `placement.status === 'confirmed'`.

Landlord

Credentials, `verified`, `suspended`, `idDocument`, and the computed `trustScore` with its `trustFactors` breakdown.

Company

A SIWES placement organisation: `name`, `industry`, `address`, GeoJSON `location`, `siwesSlots`, the departments it accepts, `verified`, and `suggestedBy` for student submissions awaiting review.

Booking

The commercial record.

- Lifecycle: `pending` → `accepted` → `paid` → `movedIn` → `completed`, with `declined`, `cancelled` and `refunded` as terminal branches
- `cost` — a **frozen** subdocument. The full breakdown is copied in at creation, so a later change to the listing's price cannot alter what was agreed.
- `escrow` — `{ state, heldAt, releasedAt, refundedAt, refundReason }`
- `payment` — `{ provider, reference, amount, paidAt, raw }`
- A partial unique index prevents two live bookings on the same listing, while allowing any number of cancelled or completed ones

Supporting collections: `Admin`, `Review`, `Report`, `UserReport`, `Conversation`, `Message`, `SavedSearch`, `Comparison`, `Feedback`, `CompanyFeedback`, `LoginAttempt`.

API reference

112 endpoints across 14 route groups, all under `/api`.

Guards are shown as they apply. `protect` = signed in; `studentOnly` / `adminOnly` = role; `idVerified` = `requireIdentityVerified`.

`/api/auth` — 15 endpoints

Rate limited more tightly than the rest of the API.

METHOD	PATH	GUARDS
POST	/student/register	captcha, validation
POST	/landlord/register	captcha, validation
POST	/student/login	validation, brute-force
POST	/student/verify-otp	—
POST	/landlord/login	validation, brute-force
POST	/landlord/verify-otp	—
POST	/admin/login	validation, brute-force
GET	/verify-email	—
POST	/verify-email/resend	—
POST	/resend-verification	protect
POST	/forgot-password	captcha, validation
POST	/reset-password	validation
POST	/change-password	protect, validation
PATCH	/tour-complete	protect
POST	/logout	—

`/api/listings` — 12 endpoints

METHOD	PATH	GUARDS
GET	<code>/</code>	—
GET	<code>/search</code>	optionalAuth
GET	<code>/cities</code>	—
GET	<code>/:id</code>	optionalAuth
GET	<code>/landlord/:landlordId/public</code>	—
GET	<code>/landlord/mine</code>	protect
POST	<code>/geocode-preview</code>	protect, idVerified
POST	<code>/</code>	protect, idVerified, validation
PUT	<code>/:id</code>	protect (ownership checked)
DELETE	<code>/:id</code>	protect (ownership checked)
PATCH	<code>/:id/availability</code>	protect
PATCH	<code>/:id/view</code>	—

`GET /search` accepts `q`, `city`, `area`, `roomType`, `minPrice`, `maxPrice`, `amenities`, `nearPlacement`, `radiusKm`, and pagination.

`/api/bookings` — 11 endpoints

METHOD	PATH	GUARDS
GET	<code>/quote</code>	—
GET	<code>/mine</code>	protect
GET	<code>/:id</code>	protect (participant only)
POST	<code>/</code>	protect, studentOnly, idVerified
PATCH	<code>/:id/respond</code>	protect (landlord)
PATCH	<code>/:id/cancel</code>	protect
POST	<code>/:id/pay</code>	protect
POST	<code>/:id/verify</code>	protect
PATCH	<code>/:id/moved-in</code>	protect (releases escrow)
PATCH	<code>/:id/refund</code>	protect, adminOnly
GET	<code>/admin/all</code>	protect, adminOnly

`/api/companies` — 21 endpoints

Public directory reads; `adminOnly` for creating, editing and bulk import; `protect` for a student setting their placement or suggesting a centre.

Key paths: `GET /`, `GET /:id`, `GET /departments`, `GET /faculties`, `GET /near-listing/:listingId`, `PUT /placement`, `POST /suggest`, `POST /bulk`, `GET /:id/feedback`.

`/api/admin` — 21 endpoints

All `protect`, `adminOnly`. Grouped as dashboard, reports and moderation, landlord verification and suspension, listing flagging and fraud rescans, student and landlord ID review, and `GET /health`.

Remaining groups

GROUP	COUNT	NOTES
<code>/api/messages</code>	7	Starting a conversation needs <code>idVerified</code> ; replying does not
<code>/api/kyc</code>	4	Submit and check ID documents
<code>/api/profile</code>	4	Includes avatar upload and account deletion
<code>/api/reviews</code>	4	Writing requires a completed stay
<code>/api/saved-searches</code>	5	All <code>protect</code> , <code>studentOnly</code>
<code>/api/saved</code>	3	All <code>protect</code> , <code>studentOnly</code>
<code>/api/comparisons</code>	3	All <code>protect</code> , <code>studentOnly</code>
<code>/api/reports</code>	1	Raise a report
<code>/api/feedback</code>	1	Product feedback

Location and geocoding

Provider: Nominatim (OpenStreetMap). Free, no API key, rate limited to one request per second, requires an identifying User-Agent.

On listing creation the address is geocoded in the background. Failure is not fatal — the listing publishes without coordinates and can be geocoded later or pinned by hand.

Precision is recorded, not just coordinates. `geocodePrecision` and `locationSource` distinguish an exact rooftop match from a street-level guess from a landlord-dragged pin. `locationConfirmedAt` marks human confirmation.

Manual pin placement is authoritative. Address lookup in Nigerian cities is unreliable, so a landlord can drag the marker. A confirmed pin is never silently overwritten by a later automated lookup.

Queries.

- `$near` / `$nearSphere` — sorted-by-distance results

- `$geoWithin` with `$centerSphere` — a radius filter

Implementation note. `$nearSphere` is illegal inside `countDocuments()`. Paginated proximity search therefore builds a parallel `$geoWithin` filter for the count. Using the same filter for both returns a 500.

Background writes must not touch `updatedAt`. Geocoding and fraud scanning save with `{ timestamps: false }`. Without it, a month-old listing rescanned in the background displays "updated less than a minute ago" — the data is right and the interface lies.

SIWES placement matching

The feature that distinguishes NestFinder from a general property site.

The directory holds 108 verified organisations in Ibadan that take industrial-training students, each with coordinates and the departments it accepts, structured around the university's faculties.

The anchor. A student sets their placement to a company. Once `status === 'confirmed'`, that company's coordinates become the origin for distance calculations, and:

- Housing search can be restricted to a radius (1–50 km) around the workplace
- Listings show a real commute distance rather than a generic estimate
- A listing page can show which placement centres are nearby

Centres not in the directory. A student submits name and address; it is geocoded so distance features work immediately for them, and an administrator reviews it before it becomes visible to everyone.

The anchor is fail-closed. A saved search anchored to a placement, whose owner no longer has a confirmed placement, **matches nothing**. It does not silently widen to the whole city. Quietly returning city-wide results would mean emailing a student about rooms nowhere near a workplace they no longer have — worse than returning nothing. The interface says the anchor is unavailable.

Comparison scoring

Up to three listings, scored across four dimensions, each weighted by the student.

DIMENSION	BASIS
Price	Normalised against the others in the comparison — cheapest scores 100
Amenities	Count relative to the best-equipped in the set
Trust	The landlord's trust score
Availability	Whether the property is currently available

Default weights are `price: 70, amenities: 60, trust: 80, availability: 90`, adjustable per student.

Scores are **relative to the set being compared**, not absolute. A listing scoring 100 on price is the cheapest of those three, not cheap in absolute terms. This is the correct behaviour for a decision between specific options, and worth stating so the number is not over-read.

Bookings and escrow

Cost breakdown. Rent × months, caution deposit, agent fee, legal fee, service fee — all shown before commitment and frozen into the booking at creation.

The split. Of the divisible amount (rent + agent fee + legal fee): **70% landlord, 5% service, 25% platform.**

The **caution deposit is held whole** and is not split. It is the student's money, refundable, and treating it as revenue would be wrong.

Escrow. Payment is held, not forwarded. Release happens only when the student confirms move-in. Before that an administrator can refund. This is the mechanism that makes a listing safe to pay for sight-unseen.

Reviews are gated on this. `Booking.hasStayedAt()` confirms a completed stay before a review can be written, which is what separates these ratings from a site where anyone can post.

Status. Payment runs through a provider seam with a sandbox implementation. Real money movement is deliberately not enabled. Before it is, a **payout queue is required**: without one the interface would state "escrow released to the landlord" when no transfer occurred. A false statement about someone's money is worse than a missing feature.

Fraud detection and trust

Listing scanning (`services/fraudShield.js`) runs on creation and on demand from the admin console:

- **Duplicate images** — photographs are fingerprinted by `utils/imageHash.js` (perceptual hashing via `sharp`) and matched against existing listings, catching the classic advance-fee rental scam where photos are lifted from a real property. The comparison is scoped to **other landlords' listings**, so a landlord legitimately reusing a photo of their own building across their own listings is not penalised.
- **Implausible pricing** — compared against `monthlyPrice` for the area
- **Text heuristics** — descriptions are normalised first, so trivial edits do not defeat duplicate detection

Output is a `fraudScore` and `fraudLevel` ; high scores set `flagged` , hiding the listing from students pending review.

Price comparison uses `monthlyPrice` , never the raw `price` . Comparing raw values mixes annual and monthly figures, and would brand an honest ₦16,000/month room as a scam.

Trust score (0–100, from a base of 40):

FACTOR	EFFECT
Identity verified	+30
Account age	+5 per month, capped at +15
Average rating	up to +15
Average fraud score	up to -40
Flagged listings	-10 each, capped at -20
Reports	-5 each, capped at -15
Suspended	-50

This feeds the trust dimension in student comparisons.

Security

CONTROL	IMPLEMENTATION
Security headers	<code>helmet</code>
CORS	Explicit origin allowlist, credentials enabled
Rate limiting	100 requests / 15 min globally; 10 / 15 min on auth
Body size	Capped at 10 kb
Parameter pollution	<code>hpp</code>
Password storage	<code>bcryptjs</code>
Session token	JWT in an httpOnly, sameSite, secure cookie
Brute force	Per-email lockout, indistinguishable response
Uploads	Type and size restricted; served with restrictive headers
CAPTCHA	Cloudflare Turnstile on registration and password reset

On injection. No dedicated sanitiser middleware is installed, deliberately. The protection is structural:

- **Query strings** — Express 5's default parser turns `?roomType[$ne]=x` into the literal key `"roomType[$ne]"`, so a Mongo operator cannot arrive that way.
- **JSON bodies** — every field is coerced at the boundary. `cleanCriteria` in the saved-search controller applies `String()`, `Number()` and `Boolean()`, and validates `roomType` against an enum. An injected object becomes the harmless literal `"[object Object]"`.
- **Regex construction** — all user input is escaped before being compiled into a `RegExp`.
- **Mongoose** casts values to their schema types.

`express-mongo-sanitize` and `xss-clean` were both evaluated and removed. Both operate by reassigning `req.query`, which Express 5 makes getter-only, so neither can run at all under this stack. `xss-clean` is additionally unmaintained.

On stored XSS. React escapes all rendered text and the codebase contains no `dangerouslySetInnerHTML`, so there is no injection point for rendered markup.

Secrets live only in `server/.env`, which is git-ignored. `server/uploads/` holds identity documents and is git-ignored except for a `.gitkeep`. No credential is written into source — `createAdmin.js` reads `ADMIN_EMAIL` and `ADMIN_PASSWORD` from the environment and refuses to run without them.

Testing

SUITE	COUNT	STACK
Server	229	Vitest, Supertest, mongodb-memory-server
Client	42	Vitest, jsdom, React Testing Library

Server tests run against an in-memory MongoDB — no external database, nothing real touched. The test setup **deletes** `EMAIL_*`, `RESEND_API_KEY` and `TURNSTILE_SECRET_KEY` from the environment before any test runs, so a suite cannot email a real person even though live credentials exist locally.

Continuous integration (`.github/workflows/ci.yml`) runs on every push: both suites on Node 20 and 22, a production build, and lint.

Guards are proven by removal. For security-critical behaviour, the guard is deleted and the test confirmed to fail before being restored. This has been done for escrow release, refund-after-release, alert throttling, the placement anchor, landlord pin protection, and background-write timestamps. A test that passes with the protection removed is not testing the protection.

Known limitation, stated plainly. The suites cover logic well and rendering barely. Two real defects shipped past a fully green suite: an entire class of invisible text caused by a colour token colliding with a Tailwind font-size utility, and a deleted `useEffect` that removed a component's data load. The first was found by looking at a screenshot; the second by a lint rule. Neither was caught by 271 passing tests.

Notable decisions

A shared filter builder. `utils/listingFilter.js` builds the query for both live search and saved-search alerts. Two implementations would drift, and the failure mode is emailing a student about a room the search would not return.

Rejecting listings with bad geocoding was considered and refused. Automated address lookup is unreliable in Nigerian cities; rejecting on that basis would exclude legitimate landlords for a fault in the geocoder. A draggable pin puts the correction in the hands of the person who knows the answer.

Annual and monthly prices are normalised, not merely stored. A derived `monthlyPrice` is what every filter, sort and fraud check reads.

External services degrade rather than block. Email, CAPTCHA, payments and error reporting all sit behind seams that no-op when unconfigured, so the application starts and demonstrates on any machine.

Real money is deliberately not enabled. See the escrow section — a payout queue is a prerequisite, not a follow-up.

Environment variables

VARIABLE	REQUIRED	PURPOSE
<code>MONGO_URI</code>	Yes	Database connection
<code>JWT_SECRET</code>	Yes	Token signing
<code>PORT</code>	No	Defaults to 5000
<code>NODE_ENV</code>	No	Enables secure cookies in production
<code>CLIENT_URL</code>	No	CORS origin and email links
<code>JWT_EXPIRE</code>	No	Token lifetime
<code>EMAIL_USER</code> , <code>EMAIL_APP_PASSWORD</code>	No	SMTP; without them, email prints to console
<code>RESEND_API_KEY</code>	No	Alternative email provider
<code>EMAIL_FROM</code>	No	Sender identity
<code>ADMIN_EMAIL</code> , <code>ADMIN_PASSWORD</code>	For <code>createAdmin.js</code>	Admin account creation
<code>TURNSTILE_SECRET_KEY</code>	No	CAPTCHA; passes through when absent